Instituto Politécnico Nacional · Escuela Superior de Cómputo

Trabajo Terminal 2027-A126 · «Prototipo de sistema para la distribución óptima de áreas de servicio en eventos masivos mediante algoritmos bioinspirados»

Reporte de implementación del algoritmo

Actividad 1: implementación del algoritmo bioinspirado, un algoritmo genético multiobjetivo con ordenamiento por no dominancia sobre las dos funciones objetivo

Periodo 13 al 26 de octubre de 2026
Responsable Martínez Meneses Monserrat Alejandra
Integrantes Echeverría Pérez Luis David · Martínez Meneses Monserrat Alejandra
Directores Dr. Daniel Molina Pérez · Dr. Jacobo Torres Figueroa
Código https://github.com/Luis-David-Echeverria/RepoUnificadoTT (commit `6fa4c11`)

1. Objetivo

Implementar el algoritmo NSGA-II diseñado en el periodo anterior para buscar distribuciones de módulos que minimicen al mismo tiempo la distancia promedio a las áreas de servicio (F1) y el tiempo de evacuación (F2), respetando las seis restricciones del modelo. La implementación sigue las secciones 5.3 y 5.4 del reporte técnico [1].

2. Alcance

Lo que se entrega es el motor de optimización en Python, que se ejecuta desde la línea de comandos sobre recintos de prueba descritos en archivos JSON. Su propósito en TT1 es validar el modelo y medir cuánto cuesta cada corrida. La interfaz web, la API en FastAPI y la carga de recintos desde el editor forman parte del desarrollo de TT2.

3. Organización del código

Tabla 1. Módulos del motor de optimización

Archivo	Responsabilidad	Funciones principales
`parametros.py`	Parámetros de Weidmann, del SFPE, de la NOM-011 y del algoritmo	`velocidadWeidmann`
`recinto.py`	Malla de 1 m, muros y salidas en las aristas, celdas del recinto y franjas frente a las salidas	`colocarMuro`, `clasificarCeldas`, `calcularFranjas`
`modulos.py`	Módulos, giros, puertas y posiciones válidas	`rotar`, `mascaraEstatica`, `posicionesValidas`
`eikonal.py`	Fast Marching Method con muros en las aristas	`resolverEikonal`
`evacuacion.py`	Simulación de la evacuación	`simularEvacuacion`
`evaluador.py`	Violación total G, accesibilidad F1 y tiempo de evacuación F2	`violaciones`, `accesibilidad`, `evaluar`
`nsga2.py`	Algoritmo NSGA-II	`construirIndividuo`, `reparar`, `ordenarNoDominados`, `ejecutar`, `puntoRodilla`

4. Representación de una distribución

Cada módulo se codifica con un gen g = (x, y, o): la celda de la esquina inferior izquierda del módulo ya girado y el número de giros de 90° en sentido antihorario. Una distribución es un arreglo de K × 3 enteros en el que la fila j corresponde siempre al módulo j. El tipo, las medidas y las puertas de cada módulo no forman parte del gen y se guardan en el catálogo (clase `Catalogo`).

Al girar un módulo también giran sus puertas. La función `rotar` calcula, para cada una de las cuatro orientaciones, las medidas del módulo, la arista de cada puerta y su celda de acceso.

5. Evaluación de una distribución

La clase `Evaluador` cuenta primero las violaciones de las seis restricciones de la sección 5.3.5 y las suma en G. Solo si G = 0 calcula F1 con el FMM desde las celdas de acceso de los módulos de cada tipo, y F2 con la simulación de la evacuación. Así no se gasta tiempo de cómputo en distribuciones que no se pueden entregar.

Tabla 2. Implementación de las restricciones

Restricción	Cómo se cuenta
1. Dentro del recinto y sin muros que atraviesen el módulo	Celdas del módulo fuera del recinto más aristas interiores del módulo con muro
2. Sin traslape	Celdas ocupadas por más de un módulo
3. Franja libre frente a las salidas	Celdas de módulos dentro de la franja de 2 m frente a cada salida
4. Puertas utilizables	Puertas con muro encima o con la celda de acceso fuera del recinto u ocupada
5. Paso de 2 m en espacios abiertos	Puertas desde las que un cuadro de 2 × 2 m no llega a una salida, solo si en el recinto vacío sí llegaba
6. Sin celdas encerradas	Celdas libres sin camino a una salida

6. Operadores del algoritmo

Tabla 3. Etapas de NSGA-II y su implementación

Etapa	Función	Implementación
Población inicial	`construirIndividuo`	Coloca los módulos del más grande al más pequeño, cada uno en una posición y orientación válidas elegidas al azar. Si un módulo no cabe, reinicia hasta 50 veces.
Reparación	`reparar`	Recorre el cromosoma en orden. Conserva la posición heredada si es válida, y si no, busca la posición válida más cercana.
Ordenamiento	`ordenarNoDominados`, `domina`	Frentes por dominancia restringida: una distribución válida domina a una inválida, entre inválidas gana la de menor G y entre válidas se usa la dominancia de Pareto.
Diversidad	`distanciaAglomeracion`	Distancia de aglomeración dentro de cada frente.
Selección	`torneoBinario`	Gana el de mejor frente y, si empatan, el de mayor distancia de aglomeración.
Cruza	`cruzaUniforme`	Con probabilidad 0.9, cada módulo completo se toma al azar de uno de los dos padres.

Mutación	`mutar`	Cada módulo, con probabilidad 1/K, se desplaza hasta 3 celdas, se gira o se reubica al azar.
Reemplazo elitista	`ejecutar`	Se juntan padres e hijos y se conservan los mejores por frente y distancia de aglomeración.
Distribución final	`puntoRodilla`	Con los dos objetivos normalizados en el frente, la de menor suma.

Las posiciones válidas de cada módulo se calculan una sola vez por recinto (`mascaraEstatica`), considerando que quede dentro del recinto, sin muros que lo atraviesen, fuera de la franja frente a las salidas y con sus puertas utilizables. Durante la construcción y la reparación solo falta revisar los módulos ya colocados, lo que se hace con una imagen integral en `posicionesValidas`. Las evaluaciones se guardan en memoria para no repetir el cálculo de una distribución que ya se evaluó.

7. Parámetros

Tabla 4. Parámetros de NSGA-II en las primeras corridas

Parámetro	Valor	Origen
Tamaño de la población (P)	40	Elegido para que una corrida tome alrededor de medio minuto
Generaciones (G_{max})	50	Igual que el anterior
Probabilidad de cruza (p_c)	0.9	Valor usado por Deb et al. [2]
Probabilidad de mutación por módulo (p_m)	1/K = 1/6	Valor usado por Deb et al. [2]
Desplazamiento máximo en la mutación (r)	3 celdas	Diseño

8. Verificación

Se generaron 200 distribuciones por recinto de dos formas: colocando los módulos completamente al azar y con la construcción de la población inicial. La Tabla 5 muestra qué porcentaje cumplió las seis restricciones, el costo medio de una evaluación completa y el costo de las corridas del periodo, descritas en el reporte de las primeras corridas.

Tabla 5. Distribuciones válidas y costo de evaluación

Recinto	Válidas al azar	Válidas construidas	ms por evaluación	Evaluaciones por corrida	Segundos por corrida
Explanada abierta	9.5 %	57.0 %	13.2	1,569	34.6
Plaza con escenario	3.5 %	57.5 %	11.7	1,685	30.8
Salón con muro divisorio	12.5 %	100.0 %	20.3	1,651	38.9

Colocar los módulos completamente al azar solo produjo entre 3.5 % y 12.5 % de distribuciones válidas. Con la construcción de la población inicial (sección 5.4.2) el porcentaje sube a 57.0 % o más en los recintos abiertos y a 100.0 % en el cerrado. Las distribuciones construidas que no son válidas fallan únicamente en la restricción 5, el paso de 2 m hacia las salidas, que la construcción

no revisa porque depende de todos los módulos a la vez. Esas distribuciones se ordenan por su violación total, y en las 30 corridas del periodo toda la población quedó formada por distribuciones válidas desde la primera generación. La Figura 1 muestra la distribución del punto de rodilla obtenida en una corrida de cada recinto.

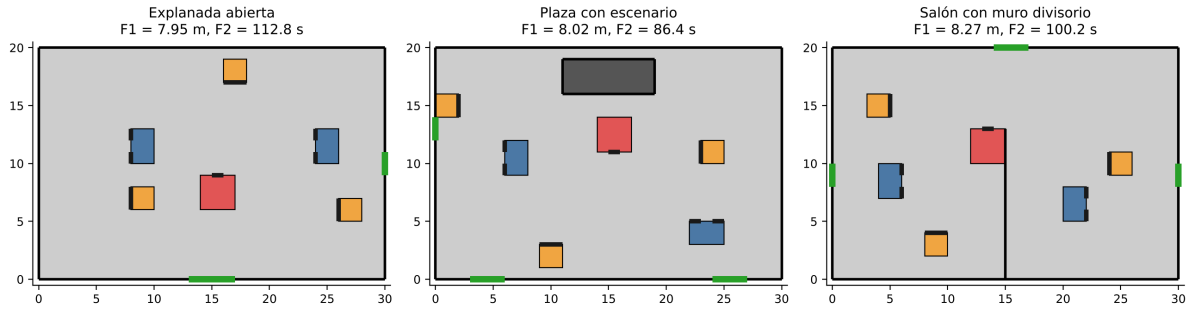


Figura 1. Distribución del punto de rodilla en cada recinto. Azul: sanitarios. Rojo: atención médica. Amarillo: puestos de comida. Las líneas gruesas negras sobre los módulos son sus puertas y las verdes, las salidas.

9. Ejecución

```
pip install -r requirements.txt
python verificacion.py
python corridas.py --semillas 10 --poblacion 40 --generaciones 50
python analisis.py
```

10. Siguientes pasos

En TT2 el motor se conectará con la interfaz web mediante la API, se ajustarán P y G_{max} con más corridas y recintos de mayor tamaño, y se evaluará en paralelo cada generación para reducir el tiempo de espera del usuario.

Referencias

[1] L. D. Echeverría Pérez y M. A. Martínez Meneses, «Prototipo de sistema para la distribución óptima de áreas de servicio en eventos masivos mediante algoritmos bioinspirados», Documento técnico de Trabajo Terminal 2027-A126, ESCOM, IPN, 2026.

[2] K. Deb, A. Pratap, S. Agarwal y T. Meyarivan, «A fast and elitist multiobjective genetic algorithm: NSGA-II», *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.